作業三｜物聯網攻擊實作與防禦方法探討

IoT Security Lab V2.2｜繁體中文最終報告＋PowerShell 實作證據

提交版本：2026-10-31｜Repository baseline: d314a283ffef79050dc5a358003caaafa13a710c

實驗	正式最終 Run	Fresh Demo Run	結果
E1｜ARP Spoofing / MITM	20261030T193936Z	20261031T061521Z	PASS
E2｜Replay Attack	20261030T193709Z	20261031T061513Z	PASS
E3｜SQL Injection	20261030T193723Z	20261031T070851Z	PASS

最終測試：27 passed，0 failed，0 skipped

說明：本報告保留原本正式 evidence baseline；PowerShell 圖片來自同一 Git commit 的 Fresh Demo Run，用來滿足課堂「實際操作畫面」要求並再次驗證可重現性。

1. 研究目的與核心概念

物聯網安全不是只有裝置本身是否安全，而是一條由裝置、網路、傳輸、應用程式到資料庫所構成的信任鏈。不同攻擊發生在不同的 Trust Boundary，因此單一控制無法處理所有風險。本作業以三個實驗分別驗證網路層、訊息層，以及應用／資料層的安全失效與防禦方式。

```
Device → Network → Transport → Message/Application → Data
          E1                  E2                   E3
```

實驗	主要問題	防禦核心
E1 ARP/MITM	網路路徑被中間人劫持	TLS + Certificate Validation；概念性控制含 DAI / Segmentation
E2 Replay	合法 Request 被重放	HMAC + Nonce + Timestamp
E3 SQLi	輸入資料被當成 SQL Code	Parameterized Query + Least Privilege

2. 實驗版本與執行環境

為避免使用到不同版本的程式，Fresh Demo 從 GitHub 重新 clone，並 checkout 到最終 closure commit。工作樹為 clean，確保以下截圖來自固定版本。

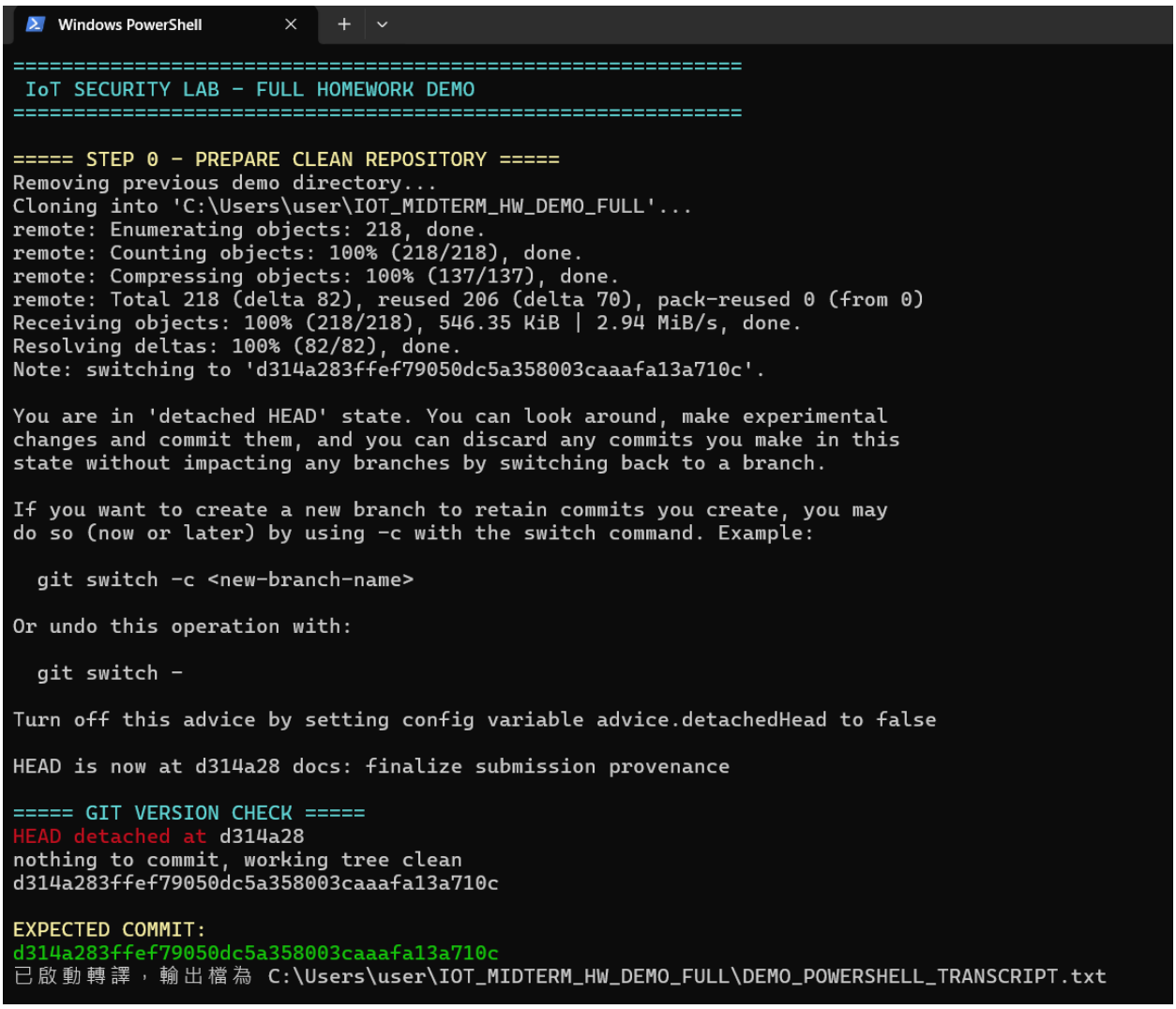


圖 1　GitHub 版本確認：checkout 至 d314a28，working tree clean。

主要執行環境如下：Python 3.14.2、Docker Engine 29.6.1、Docker Desktop 4.81.0、Docker Compose v5.2.0。

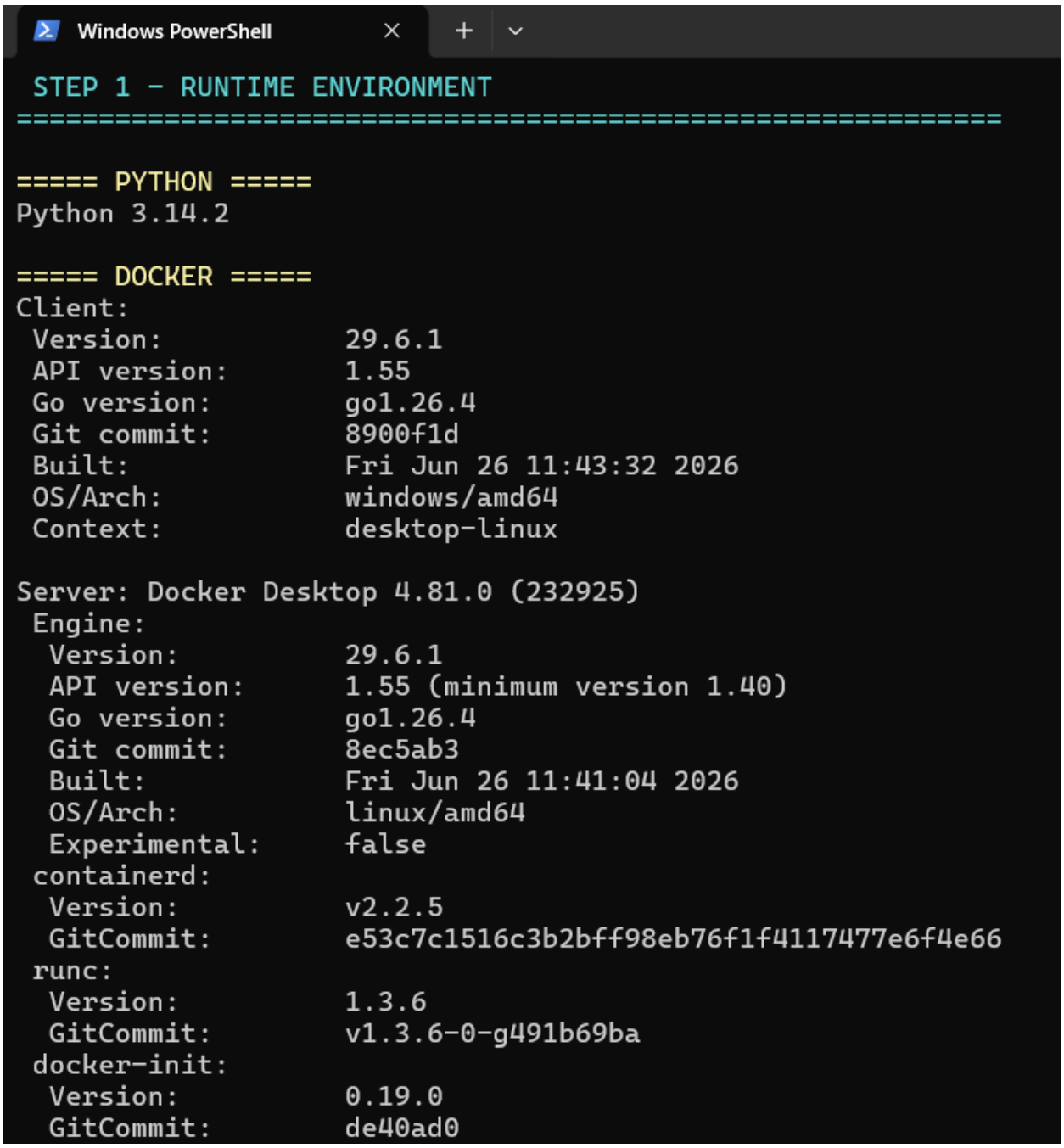


圖 2　Fresh Demo 的 Python 與 Docker 執行環境。

3. E1 | ARP Spoofing / MITM

E1 是本作業的主要攻擊。實驗在隔離的 Docker L2 網路中建立 Victim、Gateway、Attacker 三個節點，驗證 ARP Mapping 被竄改後，流量是否真的經過攻擊者，以及 HTTP 與 HTTPS 在同一 MITM 路徑下的差異。

3.1 為什麼 ARP Spoofing 能成立？

ARP 的工作是把 IP Address 對應到 MAC Address，但傳統 ARP 缺乏強式身分驗證。攻擊者可以送出偽造 ARP Reply，使 Victim 將 Gateway IP 對應到 Attacker MAC。

攻擊前：`10.77.0.2 → Legitimate Gateway MAC`
攻擊後：`10.77.0.2 → Attacker MAC`

只要攻擊者成功取得轉送路徑並開啟 IP forwarding，Victim 仍可與 Gateway 通訊，攻擊者便形成真正的 Man-in-the-Middle。若攻擊者不 forwarding，結果只是服務中斷（DoS），不能稱為 MITM。

3.2 Fresh Demo 實際結果

Fresh Demo 中 E1 的 feasibility gate、MITM path、DoS contrast、HTTP payload exposure、HTTPS payload protection、TLS certificate validation 與 regression 全部 PASS。

```
===== E3 LATEST DEMO RUN =====
C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\evidence\e3\20261031T061520Z

===== E3 CORE RESULT =====
E3-VULN-SEMANTICS                   PASS
E3-FIXED-PARAMETERIZED              PASS
E3-CONTAIN-ALLOWED-SELECT           PASS
E3-CONTAIN-SENSITIVE-READ           PASS
E3-CONTAIN-UNAUTH-UPDATE            PASS
E3-CONTAIN-UNAUTH-DROP              PASS
E3-DB-STATE                         PASS
E3-REGRESSION                       PASS

===== E3 KEY LEARNING =====
NORMAL QUERY              : 1 ROW
SQL INJECTION             : 4 ROWS
PARAMETERIZED INJECTION   : 0 ROWS
NORMAL PARAMETERIZED QUERY: 1 ROW

Least Privilege:
Sensitive SELECT = DENIED
Unauthorized UPDATE = DENIED
Unauthorized DROP   = DENIED

============================================================
 STEP 5 - E1 ARP SPOOFING / MITM
============================================================
[e1] building + starting isolated lab ...
=== E1 ARP/MITM | run 20261031T061521Z | feasibility=PASS | verdict PASS ===
  [PASS] E1-FEASIBILITY-GATE: SAME_L2_DOMAIN=YES ARP_TABLE_VISIBLE=YES ARP_MAPPING_CAN_CHANGE=YES ATTACKER_IP_FORWARDING=YES TRAFFIC_CONTINUES_THROUGH_ATTAC
KER=YES PCAP_EVIDENCE_AVAILABLE=YES
  [PASS] E1-MITM-PATH: baseline[10.77.0.2              ether   5e:bc:02:22:79:e3   C                     eth0] poisoned[10.77.0.2              ether   e
a:82:b2:d3:74:51   C                     eth0] ip_forward=1
  [PASS] E1-DOS-WITHOUT-FORWARDING: traffic_broken=True (response empty=True)
  [PASS] E1-HTTP-PAYLOAD-EXPOSED: token_in_http_pcap=True
  [PASS] E1-HTTPS-PAYLOAD-PROTECTED: token_in_https_pcap=False valid_response=True
  [PASS] E1-TLS-CERT-VALIDATION: with_ca_exit=0 without_ca='000 rc=60'
  [PASS] E1-REGRESSION: http_ok=True tls_ok=True arp_restored=True
Evidence: C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\evidence\e1\20261031T061521Z

===== E1 LATEST DEMO RUN =====
C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\evidence\e1\20261031T061521Z
```

圖 3　Fresh Demo 執行結果：E3 核心結果與 E1 Runner 的全項 PASS；E1 包含 feasibility、MITM、HTTP/HTTPS、TLS 與 regression。

ARP Table 的實際變化如下：攻擊前 Gateway IP 10.77.0.2 對應合法 MAC 5e:bc:02:22:79:e3；攻擊後同一 IP 被改綁到 ea:82:b2:d3:74:51。這是 ARP Spoofing 成功最直觀的證據。

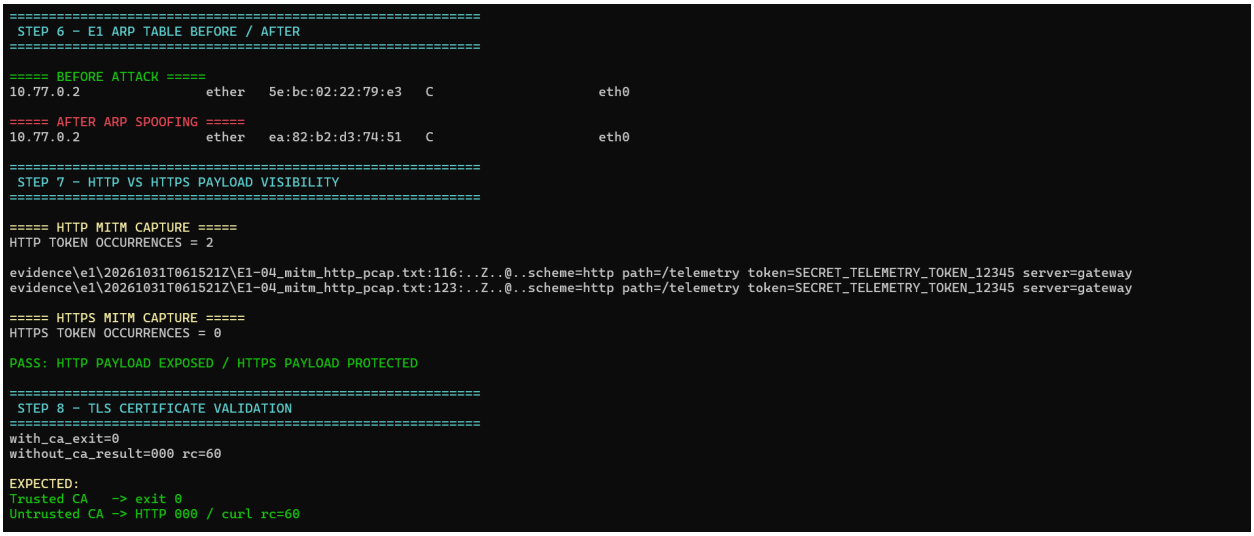


圖 4　E1 核心操作證據：ARP Mapping 前後改變、HTTP Token 可見、HTTPS Token 不可見，以及 TLS 憑證驗證結果。

3.3 HTTP 與 HTTPS 的差異

在同一個 MITM 路徑上，HTTP capture 中測試用 SECRET_TELEMETRY_TOKEN_12345 出現 2 次；HTTPS capture 中出現 0 次。這代表 TLS 並沒有阻止 ARP Spoofing，也沒有修復被劫持的 forwarding path，但能在 hostile network path 上保護 application payload 的 confidentiality 與 integrity。

3.4 Certificate Validation

Trusted CA 情境下連線成功（exit 0）；Untrusted CA 情境下 curl rc=60，HTTP code 顯示 000。000 不是 Server 回傳的 HTTP status，而是 TLS handshake 在進入 HTTP Response 前就因憑證驗證失敗而終止。

3.5 Recovery / Regression

攻擊停止後，ARP Mapping 被恢復，HTTP 與 HTTPS 均可正常使用。這證明防禦與復原並非單純「封鎖一切」，而是在恢復可信網路狀態後維持合法服務。

```
============================================================
 STEP 9 - E1 RECOVERY / REGRESSION
============================================================

===== RESTORED ARP =====
10.77.0.2                ether   5e:bc:02:22:79:e3   C                     eth0

===== HTTP AFTER RESTORE =====
scheme=http path=/ token=SECRET_TELEMETRY_TOKEN_12345 server=gateway

===== HTTPS AFTER RESTORE =====
scheme=https path=/ token=SECRET_TELEMETRY_TOKEN_12345 server=gateway

EXIT=0

============================================================
 STEP 10 - COMPLETE TEST SUITE
============================================================
...........................
27 passed in 0.80s

============================================================
 FINAL DEMO SUMMARY
============================================================

Repository Commit:
d314a283ffef79050dc5a358003caaafa13a710c

E1 DEMO RUN:
20261031T061521Z
  ARP Spoofing / MITM       = PASS
  HTTP Payload Exposure     = PASS
  HTTPS Payload Protection  = PASS
  TLS Cert Validation       = PASS
  Regression / Restore      = PASS

E2 DEMO RUN:
20261031T061513Z
  Vulnerable Replay         = SUCCESSFULLY DEMONSTRATED
  Protected Replay          = BLOCKED
  Modified Requests         = BLOCKED

E3 DEMO RUN:
20261031T061520Z
  SQL Injection             = SUCCESSFULLY DEMONSTRATED
  Parameterized Query       = PASS
```

圖 5　E1 Recovery / Regression 與完整測試：ARP 恢復、HTTP/HTTPS 恢復，以及 27 項測試全數通過。

4. E2 | Replay Attack

Replay Attack 的核心不是破解加密，而是把一份原本合法的 Request 原封不動再送一次。若 Server 只判斷 Request 是否格式正確或來源合法，而沒有判斷「這是不是第一次、是不是現在的 Request」，就可能再次執行同一動作。

```
Vulnerable：first request = 1 → replay = 2
Protected：first request = 1 → exact replay = 401 → action_count 保持 1
```

4.1 防禦機制

控制	目的
HMAC-SHA256	驗證訊息來源與內容是否遭修改
Nonce	保證一次性；已使用 nonce 不可再次接受
Timestamp	限制 Request 可接受的時間範圍
Per-Device Secret	避免整個 fleet 共用同一把 secret

Client 對實際傳送的 raw body bytes、timestamp 與 nonce 進行簽章；Server 必須驗證收到的 exact bytes，而不能 parse JSON 後重新 serialize，否則不同 bytes 可能被錯誤視為相同訊息。

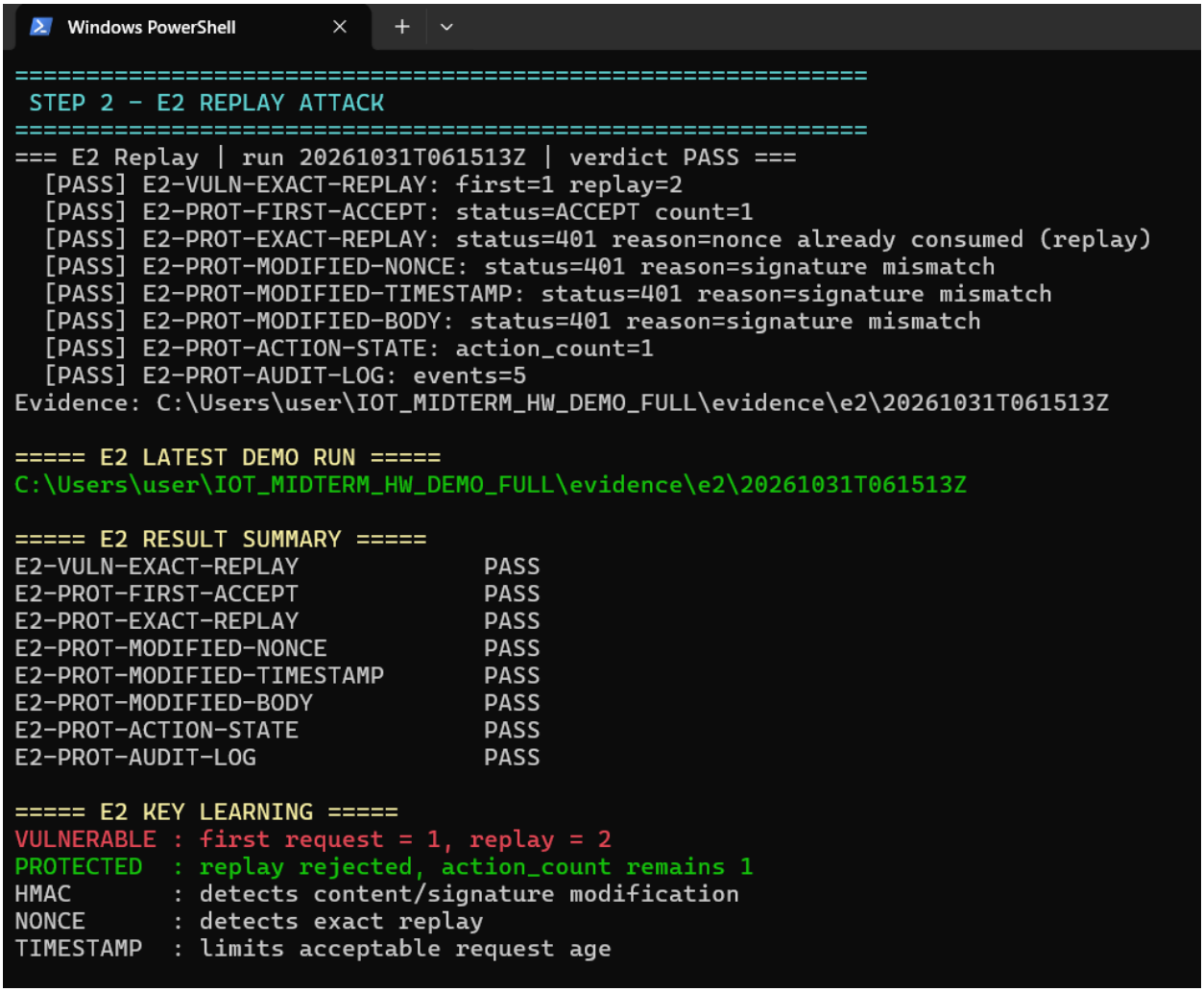


圖 6　E2 Fresh Demo：未防護時 replay 使 action_count 由 1 變 2；加入 HMAC、Nonce、Timestamp 後，完全重放與修改 request 都被阻擋。

Residual Risk：若 device secret 本身遭竊，攻擊者可以自行製造新的合法 HMAC；若 nonce cache 僅存在記憶體，Server restart 也可能造成已使用 nonce 資訊消失。因此 Replay Protection 並不等於完全解決所有身分與金鑰風險。

5. E3 | SQL Injection

E3 驗證的是 Application Data-Access / Code-Data Boundary，以及資料庫 Authorization。Runner 為了產生 deterministic evidence，直接呼叫 data-access function 與 lab PostgreSQL；HTTP transport 本身沒有被獨立驗證。

```
Normal input: sensor-alpha → 1 row
Injection: ' OR '1'='1 → 4 rows
Parameterized same input → 0 rows
Normal parameterized query → 1 row
```

5.1 Root Cause 與 Primary Fix

Raw SQL String Interpolation 讓 User Input 有機會改變 SQL 語意。Parameterized Query 則把 SQL Code 與 User Data 分開，使同一惡意字串只能被視為資料值，而不再被當成 SQL 語法執行。因此 Parameterized Query 屬於 PREVENT，也就是直接處理 SQL Injection 的 root cause。

5.2 Least Privilege

Least Privilege 不會修復 SQL Injection，但能限制 blast radius。本實驗的 iot_search_reader 可以讀取允許資料表，但 sensitive SELECT、UPDATE 與 DROP 都被資料庫權限拒絕，DB state 也維持不變。

檢查	結果
Normal query	1 row
SQL Injection	4 rows
Parameterized injection	0 rows
Sensitive SELECT	DENIED
Unauthorized UPDATE	DENIED
Unauthorized DROP	DENIED
DB state	4 → 4

為避免混用先前已存在的 PostgreSQL 容器，本次另外清除舊的 iotlab_postgres 與 iotlab_net，重新由 docker compose 建立 PostgreSQL，確認 health=healthy 後再執行 E3。Fresh Demo Run 20261031T070851Z 中，E3-VULN-SEMANTICS、E3-FIXED-PARAMETERIZED、E3-CONTAIN-* 與 E3-REGRESSION 全部 PASS。

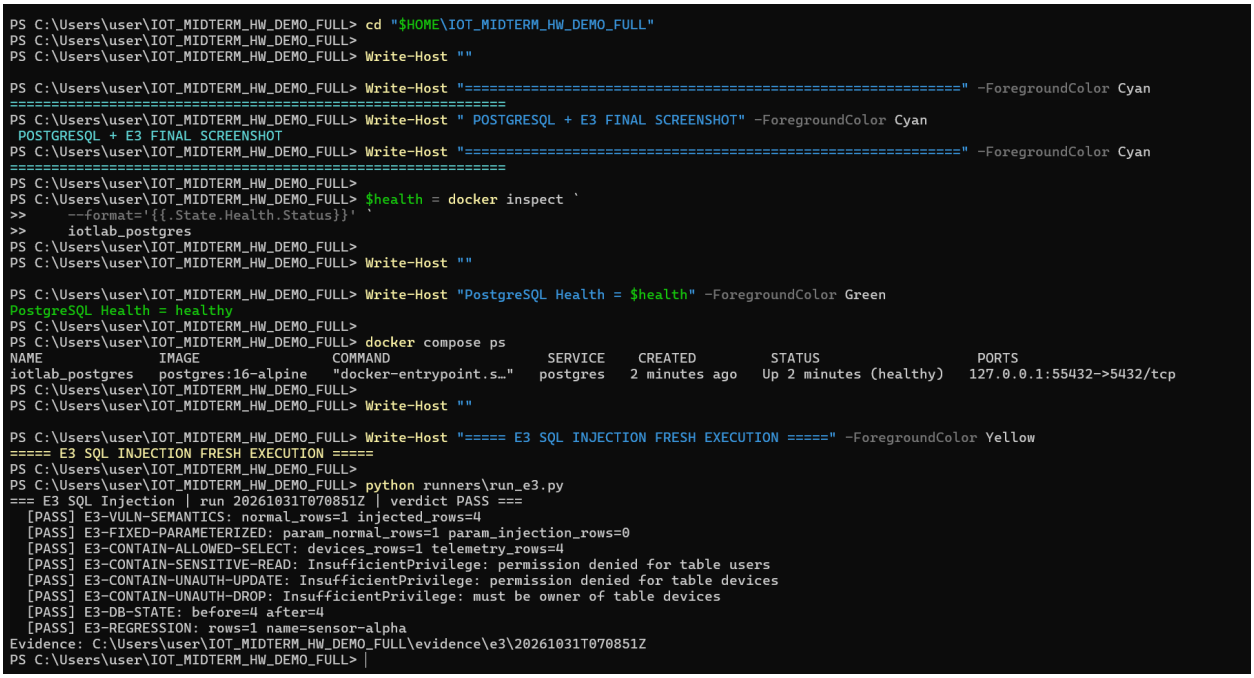


圖 7　PostgreSQL + E3 乾淨重跑：資料庫容器狀態為 healthy，SQL Injection 由正常 1 row 擴大為 4 rows；Parameterized Query 對相同惡意輸入回傳 0 rows，Sensitive SELECT／UPDATE／DROP 均被拒絕。

6. 跨層分析與 Defense-in-Depth

層級	攻擊	實作控制	重點
Network	ARP Spoofing / MITM	TLS + Certificate Validation	TLS 不防 ARP，但在 hostile path 保護 payload
Message	Replay	HMAC + Nonce + Timestamp	驗證 authenticity + freshness
Application / Data	SQL Injection	Parameterized Query + Least Privilege	Prevent root cause + Contain blast radius

三個實驗顯示，每一個安全控制都只處理特定 failure domain：TLS 無法阻止 Replay；Replay Protection 無法阻止 SQL Injection；Parameterized Query 也無法阻止 L2 攔截。因此 IoT Security 需要 Defense-in-Depth，而不是依賴單一技術。

7. Fresh Demo 最終驗證

Fresh Demo 重新從 GitHub 固定 commit 執行後，完整測試為 27 passed，且 E1、E2、E3 均再次成功重現。這些 Demo Run 不取代原本正式 evidence baseline，而是額外證明同一版程式可以在新的執行過程中再次產生相同的安全結論。

```
============================================================
 FINAL DEMO SUMMARY
============================================================

Repository Commit:
d314a283ffef79050dc5a358003caaafa13a710c

E1 DEMO RUN:
20261031T061521Z
  ARP Spoofing / MITM       = PASS
  HTTP Payload Exposure     = PASS
  HTTPS Payload Protection  = PASS
  TLS Cert Validation       = PASS
  Regression / Restore      = PASS

E2 DEMO RUN:
20261031T061513Z
  Vulnerable Replay         = SUCCESSFULLY DEMONSTRATED
  Protected Replay          = BLOCKED
  Modified Requests         = BLOCKED

E3 DEMO RUN:
20261031T061520Z
  SQL Injection             = SUCCESSFULLY DEMONSTRATED
  Parameterized Query       = PASS
  Least Privilege           = PASS
  Regression                = PASS

============================================================
 HOMEWORK DEMONSTRATION COMPLETE
============================================================
已停止轉譯，輸出檔為 C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\DEMO_POWERSHELL_TRANSCRIPT.txt

Transcript saved to:
C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\DEMO_POWERSHELL_TRANSCRIPT.txt

Demo evidence directories:
E1: C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\evidence\e1\20261031T061521Z
E2: C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\evidence\e2\20261031T061513Z
E3: C:\Users\user\IOT_MIDTERM_HW_DEMO_FULL\evidence\e3\20261031T061520Z

NOTE:
These are fresh homework demonstration runs.
They do not replace the previously submitted evidence unless you intentionally decide to update the report.
```

圖 8　Fresh Demo 最終摘要：E1 全項 PASS；E2 Vulnerable Replay 成功重現且 Protected Replay 被阻擋；E3 SQL Injection 成功重現且 Parameterized Query / Least Privilege / Regression 全部 PASS。

8. 與課堂概念的對照

課堂概念	本作業對應
CIA - Confidentiality	E1 HTTP Token 暴露；HTTPS/TLS 保護 Payload
CIA - Integrity	E1 ARP Mapping 被竄改；E2 HMAC 驗證訊息完整性
CIA - Availability	E1 不 forwarding 時形成 DoS
PKI / Certificate	E1 Trusted CA 成功、Untrusted CA 被拒絕
Authentication / Message Integrity	E2 HMAC
Freshness	E2 Nonce + Timestamp
Access Control / Least Privilege	E3 DB Role 限制
Defense-in-Depth	三個 Trust Boundary 各自配置不同控制

9. 研究限制與殘餘風險

- 所有攻擊均在受控、隔離的教育實驗環境中進行，不攻擊公開目標或第三方設備。
- E1 的 TLS 只能保護 payload confidentiality/integrity，不能解決 network availability 或 endpoint compromise。
- E2 若 device secret 洩漏，HMAC 將無法阻止持有金鑰的攻擊者建立新 request；in-memory nonce cache 也有 restart 風險。
- E3 的 Least Privilege 僅限制 blast radius，不能取代 Parameterized Query。
- E3 並未獨立驗證 HTTP transport，實驗重點是 data-access logic 與 DB authorization。

10. 結論

本作業透過 E1、E2、E3 三個實驗實際驗證 IoT Security 的跨層特性。E1 顯示 ARP Spoofing 可以改變 L2 forwarding path，而 TLS 在該 hostile path 上保護 payload；E2 顯示合法訊息仍可能被 replay，因此需要 HMAC、Nonce 與 Timestamp 驗證 authenticity 與 freshness；E3 顯示 SQL Injection 的 root cause 必須以 Parameterized Query 修正，再以 Least Privilege 限制可能損害。整體結論是：IoT Security 必須以跨層 Defense-in-Depth 管理不同 Trust Boundary 的風險。

11. 參考資料

- 課堂教材：IoT Application Information Security & Risk Management。
- Assignment #3 課堂投影片。
- RFC 826 - Address Resolution Protocol (ARP)。
- RFC 2104 - HMAC: Keyed-Hashing for Message Authentication。
- RFC 8446 - TLS 1.3。
- OWASP Top 10 - A03:2021 Injection。
- OWASP SQL Injection Prevention Cheat Sheet。
- PostgreSQL Documentation - GRANT / Role Privileges。
- CWE-294 Replay Attack、CWE-89 SQL Injection。